

Introduction

Group Name: Chambers St. Halal Food

Project Title: Draw It!

By Alwin Ng & Tahmid Islam

Period 2 APCS

Description

Our project is a single player timed drawing game. The gameplay is simple: pick an item, and draw it as fast as you can. The catch is: it's not other players guessing what you're drawing, it's a specialized AI agent trained on the Quick, Draw! database.

Functionalities:

- Canvas that interacts with your mouse
- 300+ items to choose from
- An AI agent tailored to recognize items in the game
- A point system that rewards more points the faster the agent guesses

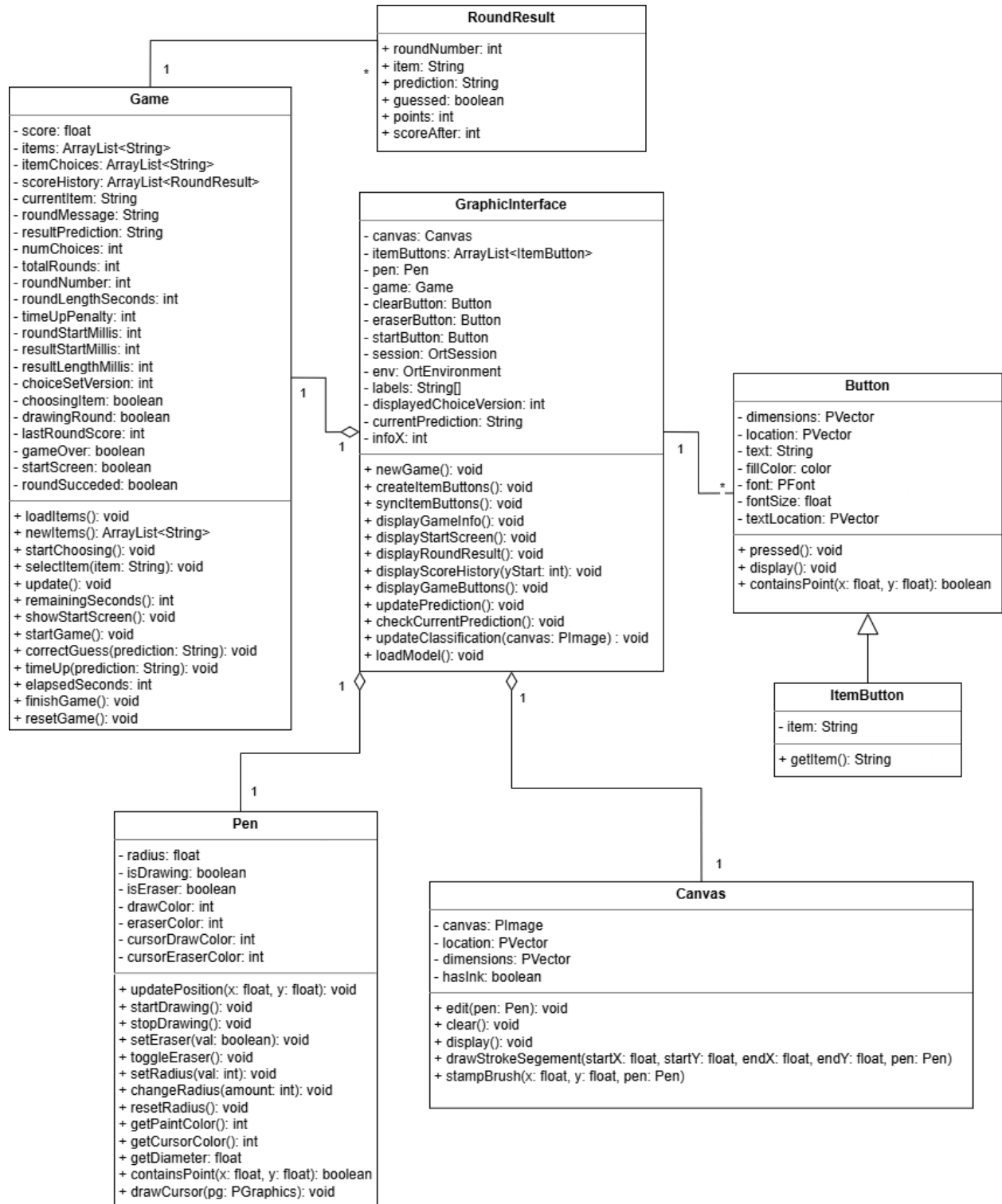
Required Libraries:

- Onnx: a library that allows us to run a local CNN model for the game

How Does It Work?

To play the game, just press the start button. The program will start the first round, where the user will select one item out of many. Then, the user will start drawing. After a certain cooldown, our AI model will start guessing out of the choices the user was given. Once the AI guesses correctly, the user will earn points based on the time it took for the AI to guess. The next round will start, and the same gameplay loop follows. This happens until the final round, where the final score is revealed, and you have the option to retry to beat your highest score.

UML Diagram



Functionalities & Issues

Full Functionality:

- There is a starting screen that gives basic instructions.
- You can select an item and the canvas will be unlocked.
- Full canvas functionality: draw, erase, change pen radius, clear; all the data is stored in a PImage.
- There is a clear button, eraser, changing sizes, etc. that all work.
- A timer that limits the drawing time.
- The model can guess from all of the item choices (300+ words, but the user chooses from a random selection of 5 each round).
- After the user is done, they can press R to try again.
- There is also an end screen that will show you how you did in each round along with what the word had been.

Issues we fixed:

- To implement the image recognition element, we had to be able to export the canvas as some sort of image file, something that wouldn't be possible using the elementary method of creating ellipses objects. To solve this issue, we wrote a simple algorithm that would detect the locations of the pen before using that information as well as the pen radius to edit the pixels of a PImage that would serve as our canvas.
- Before, the pen would skip many pixels when moving too fast, resulting in spotty, inconsistent drawing. This is because before the canvas was only stamping the brush at the current mouse during the draw() method, so when the mouse moved too fast, processing would skip over the space between stamped positions so the line would look inconsistent. We changed the pen to remember its old position so that the canvas can draw a kind of interpolated drawing between the previous and current points, ensuring that it doesn't skip over anything. Also we moved the drawing part into mouseDragged() so that each drag gets drawn immediately rather than waiting for the next frame to happen, so that the mouse movement would be able to keep more of its path, with the interpolation we mentioned earlier covering any remaining gaps.
- Creating and implementing an image recognition model was really painful, as most external APIs required the user to buy credits for tokens. To solve this, we trained a local model in Python before transferring the details to an Onnx file that is readable by Processing via Onnx's java library.

Log/Individual Contributions

Tahmid:

- Wrote the Pen class with size and eraser functionality to be able to edit directly on the canvas class, also helped write and edit syntax on the canvas and GraphicInterface.
- Implemented the timer as well as the word selection for the player.
- Added the 5 round and scoring system, improved UI and instruction guide on controls, including starting and end screen

Alwin:

- Trained the model on his computer, running a simple python script to generate an Onnx file that we will use for the Processing model.
- Wrote the initial framework for the canvas and drawing on the canvas
- Created the class structure of the project, planning with the initial UML diagram
- Also made UI fixes, bug testing, and touched up the final score system.